**Steps to Integrate GitHub**

This is clean, production-style, fully through .env, no manual remote editing, no interactive prompts.
We can use GitHub username, not email, for authentication

**GitHub username**, not your email, for authentication.
So:

```
GITHUB_USERNAME=pankaj8blr
GITHUB_TOKEN=ghp_XXXXX
```

Email is NOT used for push authentication.

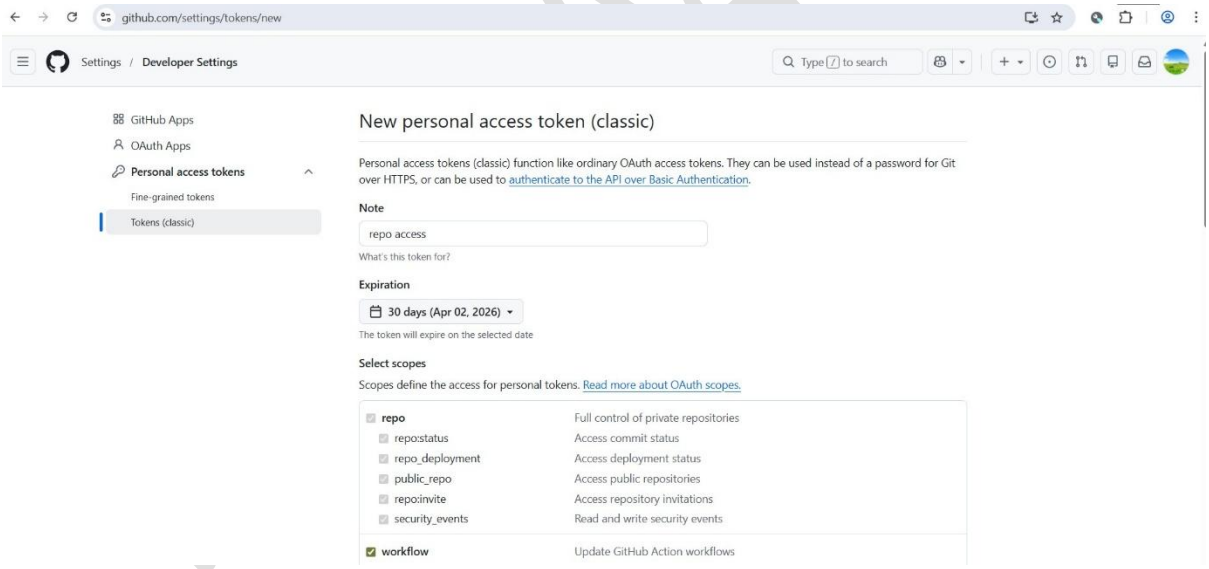
**STEP 0 : First: Create a  Token if its not present**

Since it was exposed:

1. GitHub → Settings
2. Developer settings
3. Personal access tokens
4. Generate a new one

Give it:

- Repo → Full control (private repos if needed)




Copy the new token safely.

**STEP 1 : Update .env File**

Open your project root .env file (same location as docker-compose.yml).
Add:

```
GITHUB_USERNAME=pankaj8blr
GITHUB_TOKEN=ghp_NEW_SECURE_TOKEN
```

Save file.
Do NOT commit .env to GitHub.

**STEP 2 : Changes required in git_tool.py (Proper Non-Interactive Auth)**
Open:
tools/git_tool.py

```
import git
import os

def apply_patch_and_commit(repo_path, file_path, new_content, branch_name):
    repo = git.Repo(repo_path)

    # Configure identity
    repo.git.config("user.email", "ai-agent@local")
    repo.git.config("user.name", "AI-Agent")

    # Checkout or create branch
    if branch_name not in repo.heads:
        repo.git.checkout("-b", branch_name)
    else:
        repo.git.checkout(branch_name)

    # Write updated content
    full_path = os.path.join(repo_path, file_path)
    with open(full_path, "w") as f:
        f.write(new_content)

    repo.git.add(file_path)
    repo.index.commit(f"AI update: {file_path}")

    # Inject token into remote URL dynamically
    username = os.getenv("GITHUB_USERNAME")
    token = os.getenv("GITHUB_TOKEN")

    remote_url = f"https://{username}:{token}@github.com/{username}/java-design-
patterns.git"

    repo.git.remote("set-url", "origin", remote_url)

    repo.git.push("origin", branch_name)
```

Important:

- No interactive login
- Token injected at runtime
- Clean push

**STEP 3 : Rebuild Container**

After editing file and .env:

```
docker compose down
docker compose build --no-cache
docker compose up -d
```

Wait until worker is ready.

**STEP 4 : Verify .env Loaded Into Container**

Enter worker:

docker compose exec worker bash

Then:

```
echo $GITHUB_USERNAME
echo $GITHUB_TOKEN
```

You should see:

```
pankaj8blr
ghp_xxxxxx
```

If blank → .env not mapped properly.

Make sure docker-compose.yml contains:

```
env_file:
  - .env
```

under worker service.

**STEP 5 : Confirm You're Using Your Fork**

Inside container:

```
cd /app/java_repo
git remote -v
```

It should show YOUR fork:

```
https://github.com/pankaj8blr/java-design-patterns.git
```

If not, you cloned wrong repo. Fix that outside Docker first.

**STEP 6 : Run Full Orchestrator**

Inside container:

```
python
from agents.orchestrator import run_full_agentic_flow
run_full_agentic_flow("SCRUM-5")
```

Pankaj Agrawal

**Expected Result**

Now:

✓ No username prompt

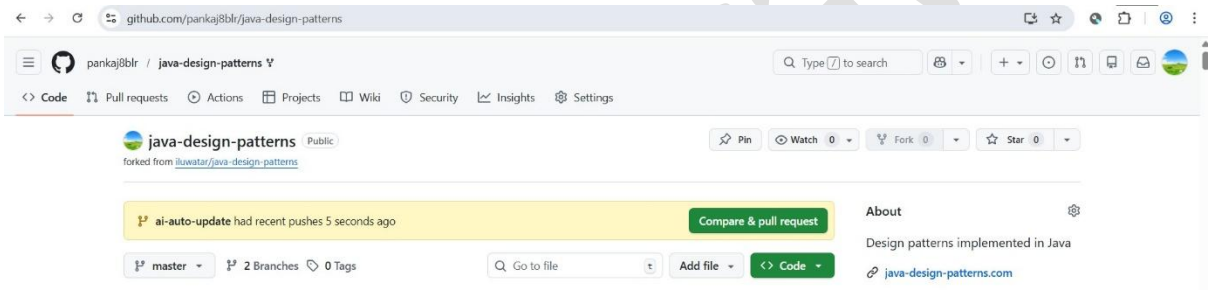
✓ No password prompt

```
PS C:\Personal\Work\Upgrad\Github\GenAI\ai-platform-JIRA-AgenticAI> docker compose exec worker bash
root@bc4d845186ed:/app# echo $GITHUB_USERNAME
echo $GITHUB_TOKEN
pankaj8blr
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
root@bc4d845186ed:/app# python
Python 3.11.14 (main, Feb 24 2026, 19:44:43) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from agents.orchestrator import run_full_agentic_flow
>>> run_full_agentic_flow("TEST-1")
{'status': 'pushed', 'branch': 'ai-auto-update'}
>>>
```

✓ Branch ai-auto-update created

✓ Commit pushed

✓ Visible in GitHub fork




**Why This Works**

We are:

- Injecting PAT dynamically
- Not relying on credential store
- Not prompting for interactive auth
- Not hardcoding in git config permanently

**Security Note**

This approach:

- Keeps token inside container only
- Reads from environment
- Does not expose via git config globally
- Safe for enterprise workflows

Pankaj Jaiswal